

# 1) Importance of Processes and Threads in Distributed Systems

## Processes and Threads in Distributed Systems

In distributed systems, processes and threads are fundamental for achieving concurrency, parallelism, and efficient resource utilization.

### Processes

- A **process** is an independent execution unit with its own memory space.
- In distributed systems, different processes may run on separate machines, communicating over the network.
- Examples:
  - A web server process handling client requests.
  - A database process handling queries.
- **Process Creation Mechanisms:**
  - **Fork-exec model:** Creates a new process by duplicating the parent process.
  - **Spawn model:** A new process is created from scratch.

### Threads

- A **thread** is a lightweight unit of execution within a process.
- Multiple threads share the same memory space, making communication faster.
- In distributed systems, multithreading is crucial for handling multiple requests simultaneously.
- **Thread Models:**
  - **User-level threads:** Managed by user-space libraries (e.g., POSIX threads).
  - **Kernel-level threads:** Managed by the OS.

## 2) OS Management in Distributed Systems

### 1. Scheduling

- The OS schedules processes/threads to optimize resource usage.
- **Types of scheduling:**
  - **Preemptive scheduling:** Uses priority levels (e.g., Round Robin, Multi-Level Queue).
  - **Non-preemptive scheduling:** A process runs until it voluntarily yields.

### 2. Concurrency

- Distributed systems rely on concurrency for efficiency.
- **Concurrency control mechanisms:**
  - **Locks (Mutex, Spinlock):** Ensures only one process/thread modifies shared data at a time.
  - **Semaphores:** Used to control access to a shared resource.
  - **Monitors:** Higher-level abstraction for synchronization.

### 3. Communication

- **Inter-Process Communication (IPC) mechanisms:**
  - **Message Passing:** Processes send/receive messages (e.g., Remote Procedure Call - RPC).
  - **Shared Memory:** Multiple processes access the same memory segment.

## Comparison of OS Architectures for Distributed Computing

| Architecture          | Description                                 | Pros                                  | Cons                                    |
|-----------------------|---------------------------------------------|---------------------------------------|-----------------------------------------|
| <b>Monolithic OS</b>  | All components run in a single kernel.      | High performance.                     | Hard to modify, not scalable.           |
| <b>Microkernel OS</b> | Minimal kernel; services run in user space. | Modular, better fault isolation.      | Higher overhead due to message passing. |
| <b>Distributed OS</b> | Manages multiple machines as one system.    | Transparency in execution and access. | Complex implementation.                 |

## 3) Distributed File Systems (DFS)

A **Distributed File System (DFS)** enables file storage and retrieval across multiple machines.

### Key Components

1. **Metadata Server:**
  - Stores file locations, access permissions, and directory structures.
  - Example: The **Master Node** in GFS.
2. **Storage Nodes:**
  - Store actual file data.
  - Files are split into chunks distributed across nodes.
3. **Caching Mechanisms:**
  - Reduce access latency by storing frequently used data.
4. **Access Control:**
  - Ensures security through permissions and authentication.

### Design Principles

1. **Transparency:**
  - Users should not need to know the file location.
2. **Scalability:**
  - Must handle large numbers of requests efficiently.
3. **Fault Tolerance:**
  - Uses replication to ensure data is available even if nodes fail.
4. **Consistency Models:**
  - **Strong Consistency:** Changes are immediately visible to all nodes.
  - **Eventual Consistency:** Updates propagate over time.

### Handling Key Issues

#### Concurrency

- DFS must handle multiple users accessing files simultaneously.
- **Mechanisms:**
  - **File Locking:** Prevents simultaneous conflicting writes.

- **Leases:** A temporary grant of file access.

## **Fault Tolerance**

- Ensures system continues working despite failures.
- **Mechanisms:**
  - **Replication:** Multiple copies of data.
  - **Checkpointing:** Saves system state periodically.

## **Replication**

- Multiple copies of data improve availability.
- Strategies:
  - **Primary-Backup:** One primary replica, multiple backups.
  - **Quorum-Based Replication:** Requires majority agreement before updates.

## **Consistency**

- Protocols like **quorum-based updates** ensure consistency.

## 4). File Service Architecture in Distributed Systems

### File Services

- A **file service** enables clients to store and retrieve files in a distributed system.

### Comparison: Stateful vs. Stateless File Services

| Feature                 | Stateful File Service           | Stateless File Service             |
|-------------------------|---------------------------------|------------------------------------|
| <b>Session State</b>    | Maintains client session info   | Does not store client session info |
| <b>Performance</b>      | Faster (caches state)           | Slower due to repeated requests    |
| <b>Failure Handling</b> | More complex recovery           | Easier recovery (resend request)   |
| <b>Scalability</b>      | Limited due to session tracking | Highly scalable                    |

### Impact on Performance, Reliability, and Scalability

- **Stateful services** improve performance but are harder to scale.
- **Stateless services** simplify recovery but increase network overhead.

## 5). Sun Network File System (NFS)

### NFS Architecture

- **Remote Procedure Calls (RPCs)** handle file operations.
- **Caching** reduces network load.
- **Authentication** uses UNIX-style access control.

### Improvements in Later Versions

- **NFSv3:** Better caching, improved read/write operations.
- **NFSv4:** Statefulness, Kerberos security.

## 6). Enhancements in Modern DFS

| Feature                | GFS                 | HDFS                  |
|------------------------|---------------------|-----------------------|
| <b>Designed For</b>    | Google Search       | Big Data              |
| <b>Fault Tolerance</b> | Master-slave        | Replication           |
| <b>Consistency</b>     | Relaxed consistency | Write-once, read-many |

## 7). Name Services in Distributed Computing

### Role of Name Services

- Resolve names to resources (e.g., DNS resolves domain names to IPs).

### DNS in Large-Scale Networks

- **Hierarchical structure** improves efficiency.
- **Load balancing** distributes requests.

## 8). Name Services vs. Directory Services

| Feature         | Name Service   | Directory Service      |
|-----------------|----------------|------------------------|
| <b>Function</b> | Resolves names | Stores structured data |
| <b>Example</b>  | DNS            | LDAP                   |

### Global Name Service (GNS) Challenges

1. **Scalability:** Uses hierarchical structures.
2. **Security:** Cryptographic authentication.
3. **Consistency:** Ensures updates are propagated.

## 9). Security and Performance in DFS and Name Services

### Security Challenges

- **Data Integrity:** Cryptographic hashing (SHA, MD5).
- **Access Control:** ACLs, Role-Based Access Control (RBAC).
- **Authentication:** Kerberos, PKI.

### Performance Challenges

- **Efficient Name Resolution:** Caching and hierarchical lookup.
- **Load Balancing:** Distributes requests evenly.
- **Replication Strategies:** Master-slave, eventual consistency models.

## 10) OS Layer and Middleware in Distributed Computing

In distributed computing, Operating System (OS) layers and middleware play crucial roles in managing resources, enabling communication, and ensuring system transparency. They provide an abstraction layer that simplifies the complexities of a distributed system, making it easier for applications to function across multiple machines.

### 1. Role of the OS Layer in Distributed Computing

The OS in a distributed system provides fundamental services to enable process execution, resource management, and inter-process communication (IPC).

#### Key Functions of the OS Layer

##### 1. Process and Thread Management

- Creates and schedules processes across distributed nodes.
- Uses multithreading for concurrent execution.
- Ensures synchronization (e.g., mutexes, semaphores) to avoid race conditions.

## 2. Memory Management

- Allocates and deallocates memory across distributed nodes.
- Supports distributed shared memory (DSM) to provide a unified memory view.

## 3. File and Storage Management

- Manages distributed file systems (DFS) (e.g., NFS, HDFS).
- Ensures fault tolerance through replication.

## 4. Communication and Networking

- Provides inter-process communication (IPC) (e.g., message passing, shared memory).
- Implements network protocols like TCP/IP and UDP.

## 5. Security and Access Control

- Enforces authentication and encryption.
- Uses Access Control Lists (ACLs) and Role-Based Access Control (RBAC).

## OS Architectures for Distributed Computing

| OS Type             | Characteristics                                       | Example                       |
|---------------------|-------------------------------------------------------|-------------------------------|
| Network OS          | Users access remote resources manually.               | Windows Server, Linux         |
| Distributed OS      | Manages multiple machines as a single system.         | Amoeba, Sprite                |
| Middleware-Based OS | Middleware handles distribution, OS remains standard. | Android (with cloud services) |

## 2. Role of Middleware in Distributed Computing

Middleware is software that sits between the OS and applications, providing abstraction, communication, and resource management.

### Key Functions of Middleware

1. Transparency (Hides the complexity of distribution)



- Access Transparency: Users interact with remote resources as if they were local.
- Location Transparency: No need to know the physical location of resources.
- Replication Transparency: Handles multiple copies of data automatically.

## 2. Communication

- Enables message passing (e.g., Remote Procedure Calls - RPC, Message Queues).
- Supports publish-subscribe models (e.g., MQTT for IoT).

## 3. Resource Management

- Load balancing: Distributes workload evenly.
- Fault tolerance: Detects and recovers from node failures.

## 4. Security

- Implements authentication (e.g., Kerberos, OAuth).
- Provides data encryption for secure transmission.

## 5. Interoperability

- Allows different systems (Windows, Linux, macOS) to communicate seamlessly.
- Uses standard APIs like CORBA, gRPC, and REST.

## Types of Middleware

| Type                              | Function                                  | Example                |
|-----------------------------------|-------------------------------------------|------------------------|
| Message-Oriented Middleware (MOM) | Asynchronous messaging between processes. | Apache Kafka, RabbitMQ |
| Remote Procedure Call (RPC)       | Enables remote function execution.        | gRPC, XML-RPC          |
| Object Middleware                 | Manages distributed objects.              | CORBA, Java RMI        |
| Database Middleware               | Facilitates distributed database access.  | ODBC, JDBC             |

|                        |                                   |                  |
|------------------------|-----------------------------------|------------------|
| Transaction Middleware | Manages distributed transactions. | IBM CICS, Tuxedo |
|------------------------|-----------------------------------|------------------|

### 3. OS vs. Middleware in Distributed Computing

| Feature              | OS Layer                                             | Middleware                                                      |
|----------------------|------------------------------------------------------|-----------------------------------------------------------------|
| Level of Abstraction | Low-level (hardware, resources)                      | High-level (services, APIs)                                     |
| Primary Function     | Manages processes, memory, files, and communication. | Provides abstraction, interoperability, and service management. |
| Examples             | Linux, Windows, Unix (with network features).        | CORBA, RPC, Kafka, gRPC.                                        |

## 11) Remote Procedure Calls (RPC) and Communication Between Distributed Objects

Remote Procedure Calls (RPC) allow programs to invoke procedures on remote systems as if they were local, abstracting network communication complexity. This is fundamental in **distributed computing**, enabling efficient interaction between distributed objects, services, and systems.

### 1. What is RPC?

- RPC is a **client-server communication mechanism** where a process on one machine can call a function that executes on another machine.
- It hides network details from the programmer, making remote calls appear just like local function calls.

#### Example Usage:

- **Microservices:** One microservice calls another over a network.
- **Distributed Databases:** A client queries a database server.
- **Cloud Services:** Applications communicate with remote APIs.

## 2. RPC Mechanism in Detail

### Step 1: Client Procedure Calls the Stub

- The **client process** makes a normal function call.
- Instead of executing locally, the call is intercepted by a **client-side stub**.

### Step 2: Marshaling the Request

- The stub **marshals** (serializes) function arguments into a format that can be transmitted over a network (e.g., JSON, Protocol Buffers).
- It also adds a **request identifier** to match responses.

### Step 3: Request Transmission

- The marshaled request is sent via the network (e.g., TCP, UDP) to the remote server.

### Step 4: Server-side Stub Receives and Unmarshals

- The **server stub** receives the request.
- It **unmarshals** (deserializes) the arguments and passes them to the actual server function.

### Step 5: Execution on the Server

- The requested function executes on the server and produces a result.

### Step 6: Marshaling the Response

- The result is marshaled into a network-compatible format.

### Step 7: Response Transmission

- The server sends the response back to the client over the network.

### Step 8: Client Stub Unmarshals the Response

- The client stub **unmarshals** the response and passes it to the original caller.
- The client function continues execution as if the remote function was executed locally.

## 3. Key Components of RPC

| Component | Function |
|-----------|----------|
|-----------|----------|

|                    |                                                              |
|--------------------|--------------------------------------------------------------|
| <b>Client</b>      | Calls the remote function.                                   |
| <b>Client Stub</b> | Prepares request (marshaling) and sends it.                  |
| <b>RPC Runtime</b> | Handles message transmission over the network.               |
| <b>Server Stub</b> | Receives request, unmarshals data, and calls the function.   |
| <b>Server</b>      | Executes the requested function and sends back the response. |

#### 4. Types of RPC Implementations

| RPC Type                | Characteristics                                              | Examples                         |
|-------------------------|--------------------------------------------------------------|----------------------------------|
| <b>Synchronous RPC</b>  | Client waits for the response before continuing.             | Traditional RPC (gRPC, XML-RPC)  |
| <b>Asynchronous RPC</b> | Client continues execution without waiting for the response. | Message queues (Kafka, RabbitMQ) |
| <b>One-way RPC</b>      | No response is sent back to the client.                      | Fire-and-forget calls            |
| <b>Secure RPC</b>       | Encrypts messages for security.                              | Kerberos-authenticated RPC       |

#### 5. Advantages of RPC

- **Transparency:** Hides network details, making remote calls appear as local.
- **Efficiency:** Optimized serialization minimizes overhead.
- **Interoperability:** Supports cross-platform communication (e.g., gRPC for polyglot services).
- **Scalability:** Enables **microservices** and **distributed systems** to communicate efficiently.

#### 6. Modern RPC Frameworks

| Framework | Description |
|-----------|-------------|
|-----------|-------------|

|                      |                                                             |
|----------------------|-------------------------------------------------------------|
| <b>gRPC</b>          | High-performance, Google-backed RPC using Protocol Buffers. |
| <b>XML-RPC</b>       | Uses XML for data encoding.                                 |
| <b>JSON-RPC</b>      | Lightweight RPC based on JSON format.                       |
| <b>Apache Thrift</b> | Supports multiple languages and efficient serialization.    |